LABORATORIO 1: ARQUITECTURA DEL COMPUTADOR

Integrantes:
- Victoria Lopez,
- Tomás Ponce
- Tomás Ordoñez

Ejercicio 1

Realizamos las siguientes modificaciones en los siguientes archivos:

Antes de comenzar con los cambios para implementar la instrucción **MOVZ** primero dejamos funcionando la pipeline. Para ello hicimos las siguientes modificaciones:

- Se implementaron las recomendaciones dadas en la guia del laboratorio
- Dado que el tamaño del programa se incrementó (fundamentalmente por los **NOP**), se aumentó el espacio de direccionamiento utilizado en **imem.sv** dado por la señal **addr** a un total de 10 bits, y junto a esto se incrementó el tamaño de la memoria rom de instrucciones en 1024 palabras, para así poder ser direccionada por los 10 bits en total. En **processor_arm.sv** pasamos los 10 bits correspondientes a **imem.sv** para poner en funcionamiento el nuevo espacio de direccionamiento.
- Utilizamos el programa "**La Olla**" para poder realizar el desarrollo más rápido. Por ende para poder correr el proyecto con el código dado se debe seguir los pasos dados por dicho programa. Es decir, armar un directorio **progs** en simulation/modelsim, colocar ahí dentro el programa en un archivo **test_code.txt** luego en el root del proyecto crear el symlink **progs** a dicho directorio creado.
  Link:
  https://famaf.aulavirtual.unc.edu.ar/pluginfile.php/27808/mod_folder/content/0/Opcode_Loader_for_LEGv8_Arch/uso%20de%20la%20OLLA.pdf

Respecto a los cambios para la implementación de MOVZ:

- En el archivo de **maindec.sv** agregamos la decodificacion de la instrucción de **MOVZ** y configuramos las señales de control según corresponde

- En **aludec.sv** agregamos 4 decodificaciones de la señal de control de **alucontrol** a partir de **aluop** y **funct**. De tal modo que dichas 4 decodificaciones correspondan a los 4 valores posibles de LSL y por ende **alucontrol** reciba 4 variantes que indican en cuál de las 4 posiciones dadas por el LSL colocar el inmediato, además de colocar todo en el registro previamente en cero. Si no se cumple ninguna de las condiciones, se establece el valor predeterminado de alucontrol en "4'b0000" como originalmente estaba.

- En **signext.sv** dada la instrucción de **MOVZ** no hacemos una extensión de signo para el inmediato sino que dejamos su valor en los 16 bits menos significativos del resultado, colocando el resto de bits en 0. Esto es para ser usado por la ALU. Si bien este cambio puede llegar a romper con el concepto de la estructura **signext.sv** dado que en si no se realiza una extensión de signo nos pareció una opción suficientemente aceptable para implementar la instrucción
- En **alu.sv** a partir de las 4 nuevas variantes de la señal de control **alucontrol** que agregamos en **aludec.sv** determinamos el resultado colocando el inmediato recibido en alguna de la 4 secciones de 16 bits que el LSL determina y colocando el resto de bits en 0.

No se realizó ningún cambio estructural, es decir, no se agregaron estructuras, ni señales de control nuevas, por ende no hicimos ningún diagrama dado que es el mismo presentado el la guía de este laboratorio.

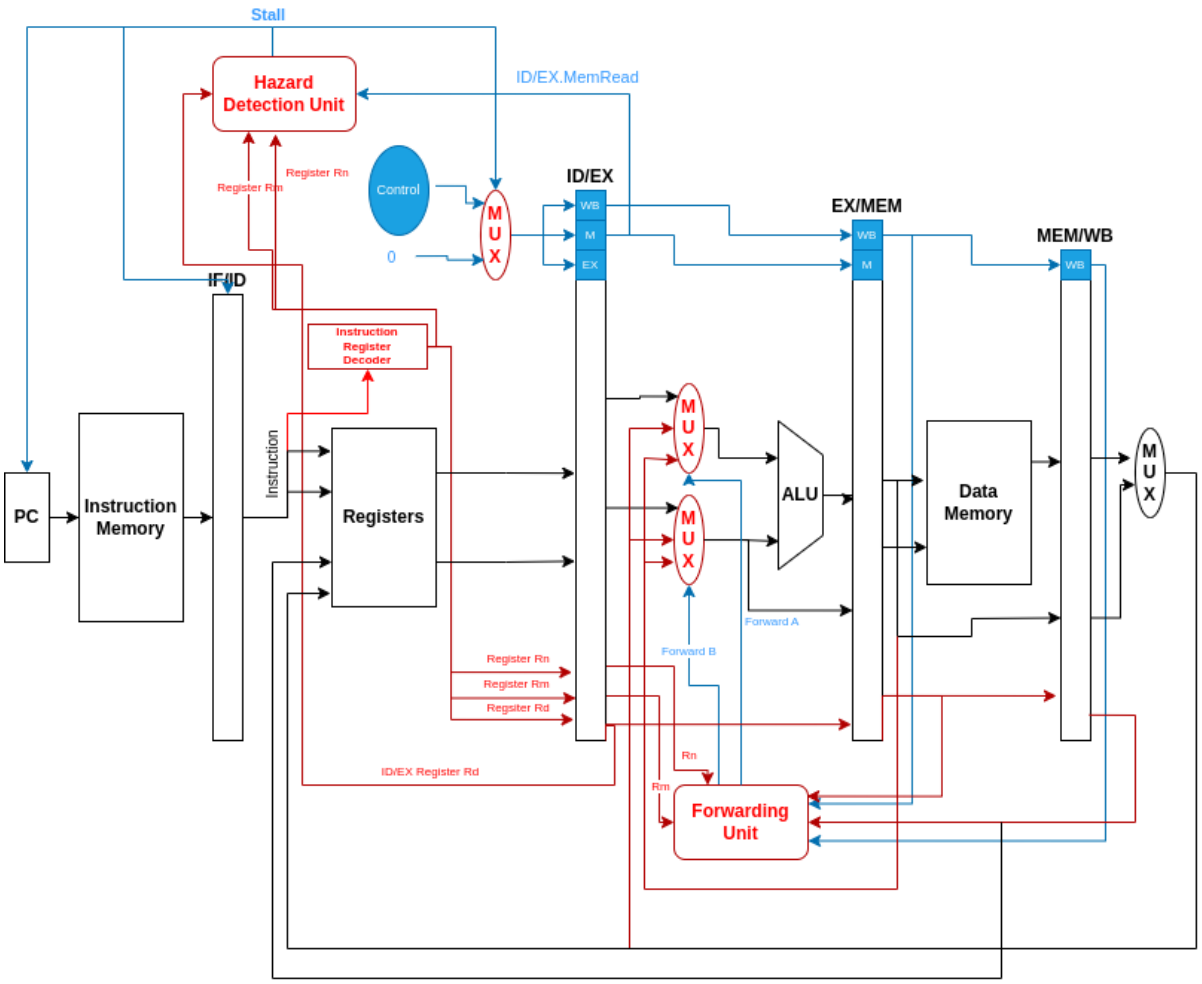
Para testear el procesador utilizamos el siguiente código:

A su vez testeamos con el código principal dado para confirmar que no dejaron de funcionar instrucciones ya hechas.

```
        .text
        .org 0x0000
        // -- Test de MOVZ
        MOVZ x29, #17, lsl 00
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
        STUR x29, [x0, #184] // Mem 23: 0x11
        MOVZ x29, #15, lsl 16
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
        STUR x29, [x0, #192] // Mem 24: 0xF_0000
        MOVZ x29, #65535, lsl 32
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
        STUR x29, [x0, #200] // Mem 25: 0xFFFF_0000_0000
        MOVZ x29, #15, lsl 48
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
        STUR x29, [x0, #208] // Mem 26: 0xF_0000_0000_0000
        // -- Fin de Test de MOVZ
infLoop:
        cbz XZR, infLoop
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
        ADD xzr, xzr, xzr // NOP
```

Ejercicio 2

Nuevo diagrama general del Procesador:



En **rojo** se muestran los nuevos módulos y conexiones realizados y en **azul** las señales de control. **Notar** cómo es un diagrama simplificado, particularmente las etapas Decode y Execute tienen mucha simplificación dado que no es necesario diagramar todo su contenido para mostrar lo realizado

Cambios Realizados:

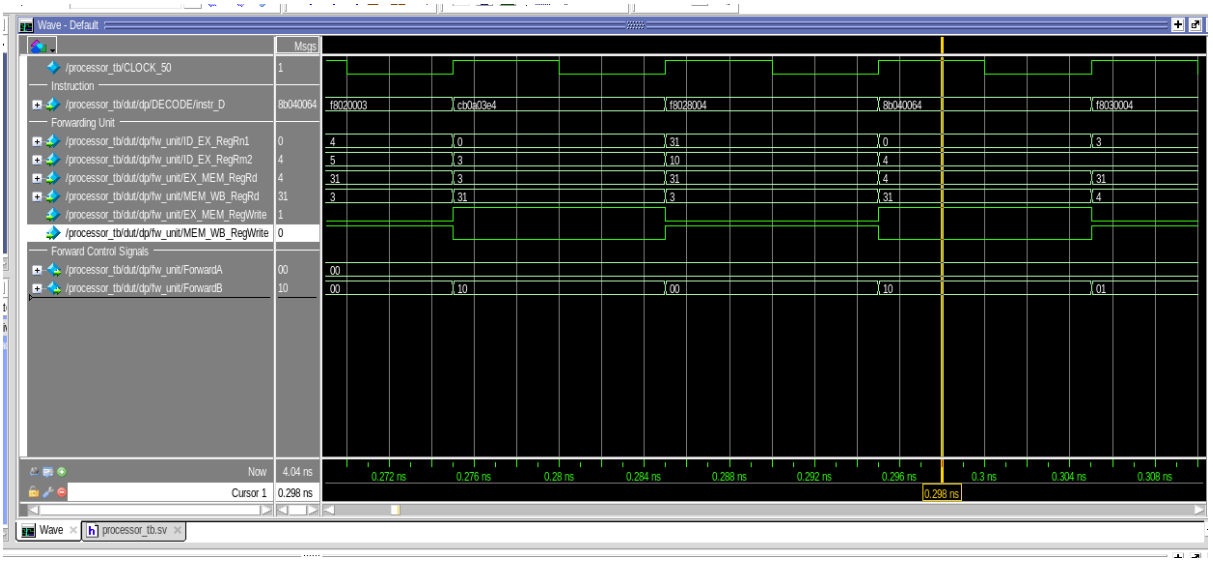
- Se agregaron los módulos de **Hazard Detection Unit**, **Forwarding Unit**, y **Instruction Register Decoder**. El hazard detection unit se encarga de detectar los hazards de datos y enviar la señal de control **Stall** para frenar momentáneamente el fecheo y procesamiento de instrucciones. La forwarding unit se encarga de hacer los distintos forwarding posibles entre los datos y el instruction register decoder se encarga de obtener los valores de los campos **Rd, Rn** y **RM** de las instrucciones que se van procesando para que la Hazard Detection Unit y la Forwarding Unit pueden detectar las condiciones correctamente
- Se agregaron las señales de control y datos correspondientes para poder realizar cada uno de estos módulos.

Wave de Forwarding:

Vamos a analizar el forward del siguiente código:

```
SUB X4, XZR, X10
STUR X4, [X0, #40] // MEM 5:0×FFFFFFFFFFFFFFF6
```

El SUB se codifica como **0xCB0A03E4** y el STUR como **0xF8028004**, podemos ver como el resultado del SUB es requerido por el STUR, y como para cuando el STUR lo necesite en la etapa execute dicho resultado va a estar recién en la etapa Memory por lo que se requiere hacer un forwarding desde Memory a Execute. La siguiente captura de señales muestra dicho forwarding:



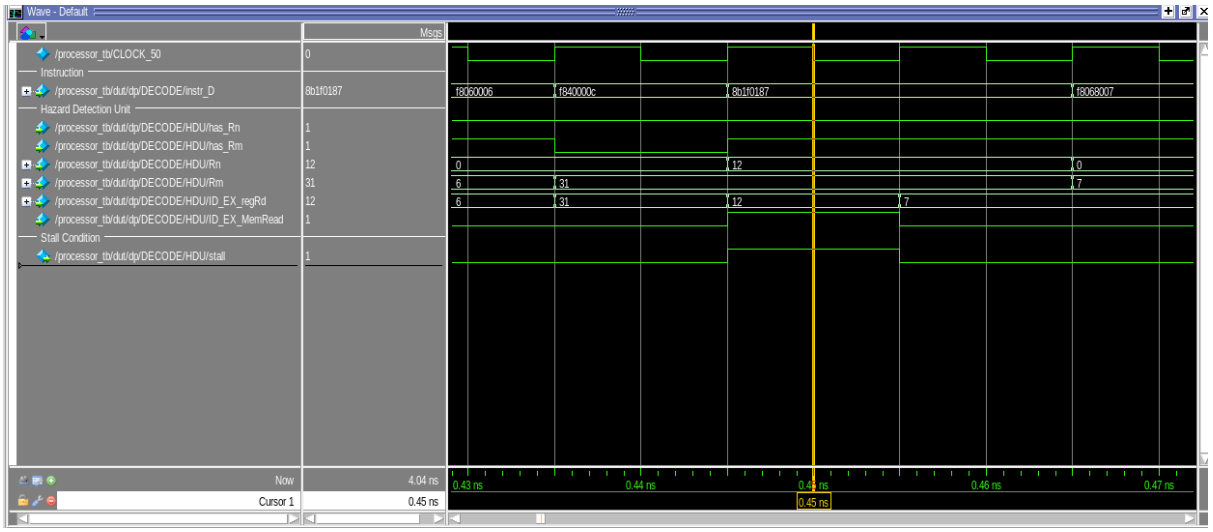
En la wave se observa cómo se decodifica el SUB, en el siguiente clock se decodifica el STUR y en el siguiente clock a ese vemos el forwarding. En dicho clock el STUR va a estar en la etapa Execute (dado que que en el clock anterior se decodificó) y el SUB en la etapa Memory por ende se da la condición de forwarding y se levanta la señal de control **ForwardB** indicando que el operando **b** de la ALU va a venir de **Ex/Mem** haciendo as el forwarding requerido.

Wave de Stall:

Vamos a analizar el stall del siguiente código:

```
LDUR X12, [X0, #0]
ADD X7, X12, XZR
```

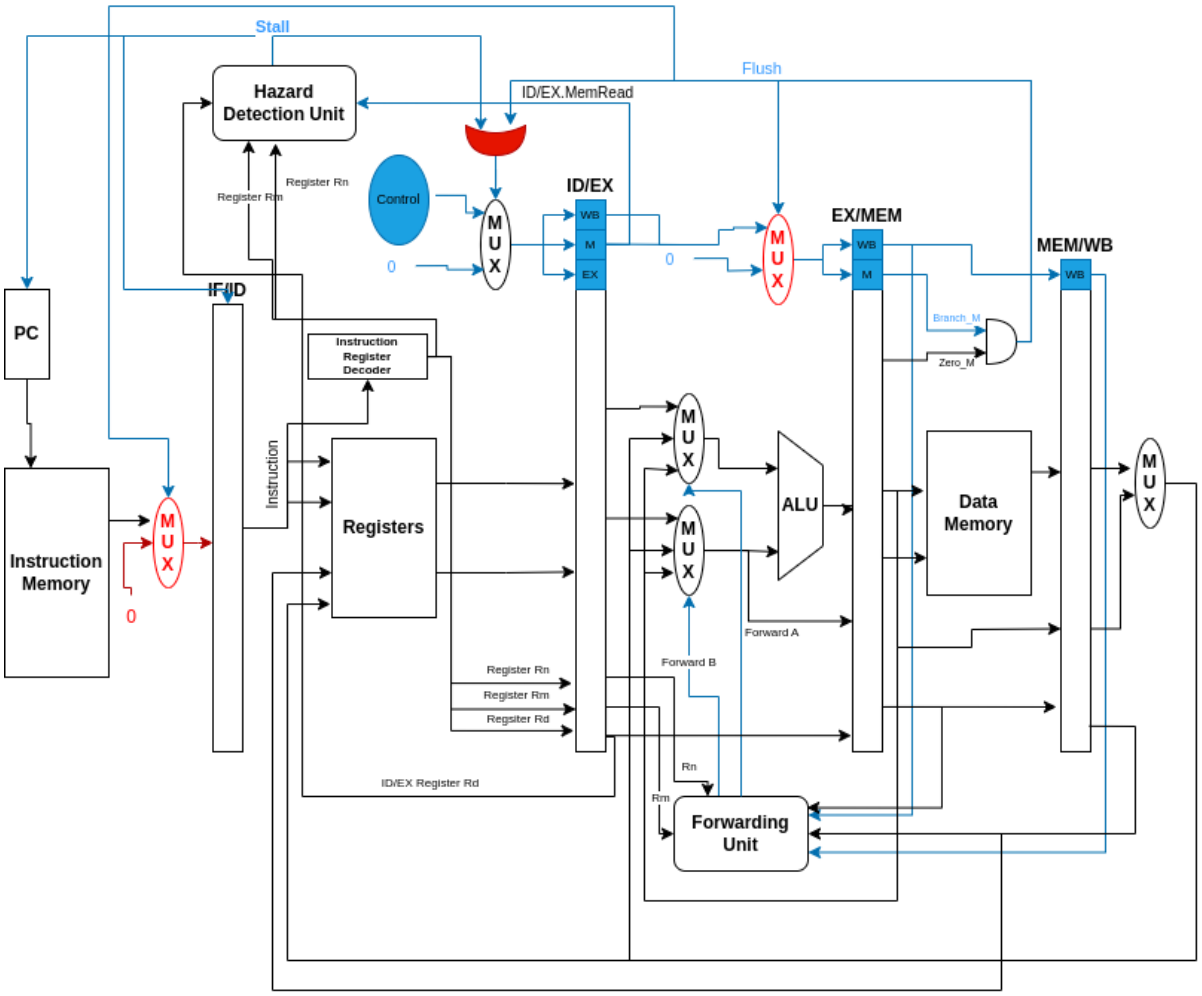
El LDUR se codifica como **0xF840000C** y el ADD como **0x8B1F0187**, podemos ver como el resultado del LDUR es requerido por el ADD, y como para cuando el ADD lo necesite en la etapa execute dicho resultado va a estar recién siendo obtenido de la memoria de datos en la etapa Memory. En este caso no se puede hacer forwarding dado que el dato aún no está disponible para cuando el ADD lo necesita. Por esto mismo es requerido hacer Stall para que se frene la ejecución del ADD e instrucciones siguientes hasta que el LDUR termine de traer el dato de memoria. El siguiente diagrama de señales muestra la condición de Stall:



En la wave se observa cómo se decodifica el LDUR, en el siguiente clock se decodifica el ADD y se levanta la condición de stall. En dicho clock el LDUR va a estar en la etapa Execute (dado que que en el clock anterior se decodificó), al hacer stall el LDUR continua a la etapa memory y el ADD se vuelve a decodificar en el siguiente clock como muestra el diagrama. Un clock siguiente a ese el ADD va a estar en la etapa execute y el LDUR en la etapa Writeback, el cual va a poder hacer forward del dato obtenido al ADD evitando así el hazard.

Ejercicio 3

Nuevo diagrama general del Procesador:



En **rojo** se muestran los nuevos módulos y conexiones realizados y en **azul** las señales de control. **Notar** cómo es un diagrama simplificado, particularmente las etapas Decode y Execute tienen mucha simplificación dado que no es necesario diagramar todo su contenido para mostrar lo realizado

Cambios Realizados:

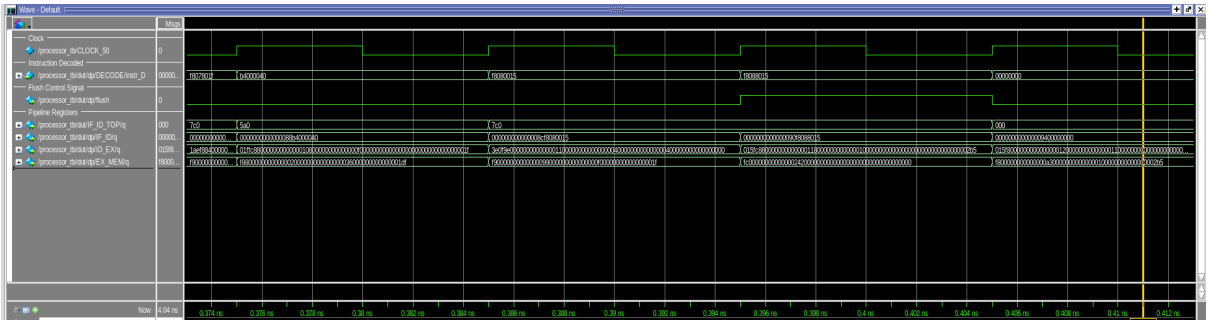
- A partir del PCSrc obtenido en la etapa Memory obtenemos la señal Flush (un simple renombre) y a partir de esta señal modificamos la entrada de los registros de pipeline para que cuando esta señal sea 1, es decir, se vaya a tomar un salto, se realice el flush colocando 0s en las señales de control de ID/EX, EX/MEM y que la instrucción fechada sea una con todos 0s.

Wave de Flush:

Vamos a analizar las siguientes instrucciones del código de prueba:

```
    CBZ X0, L1
    STUR X21, [X0, #128] // MEM 16:0×0(si falla CBZ =21)
L1: STUR X21, [X0, #136] // MEM 17:0×15
    ADD X2, XZR, X1
```

El CBZ se codifica como **0xB4000040**, el primer STUR como **0xF8080015**, el segundo STUR **0xF8088015** cómo y el ADD como **0x8B0103E2**. Si el hazard de control fuera posible entonces se debería de ejecutar erróneamente el primer STUR, dado que se va a insertar dicha instrucción en el pipeline erróneamente. La siguiente captura de señales muestra el flush de instrucciones para evitar el hazard:



En la wave se observa cómo se decodifica el CBZ, dos clock siguientes se levanta la señal de control flush, esto es porque el CBZ va a haber entrando en la etapa Memory donde se determina si el salto se debe tomar o no. Previamente, el pipeline va a tener las instrucciones siguientes al CBZ cargadas en el pipeline en las etapas Fetch, Decode y Execute. Al hacer flush se puede observar como los registros de pipeline cambian, colocándose ceros en las señales de control de ID_EX y EX_MEM, mientras que el registro de pipeline IF_ID recibe ceros correspondiente a una instrucción con todos 0. Lo anterior es difícil de mostrar a partir de la wave dado que se requiere saber en qué posición están los bits de las señales de control en el registro, esto puede saberse mirando el código. Lo más importante es que como ninguna instrucción siguiente al CBZ llegó a la etapa memory entonces el estado del procesador se mantiene y por ende se evitó el hazard de control.